

IN501

Fundamentals of Computer Programming



Unit 6 Topic

Unit Topics
Unit 1 Core Concepts
Unit 2 Creating Computer Programs: Decision Structures and Operators
Unit 3 Repetition Structures in Python
Unit 4 Manipulating Groups of Related Data Structures (tuples, lists, dictionaries)
Unit 5 Python User-Defined Functions
Unit 6 File I/O, Use of Modules
Unit 7 NumPy Third Party Library
Unit 8 Keras Third Party Library
Unit 9 TensorFlow Third Party Library
Unit 10 PyTorch Third Party Library

Unit 6 Agenda

- CSV files
- How to use Panda library
- Unit 6 module/assignment

What Is a CSV File?

- CSV stands for **Comma-Separated Values**
- A CSV file is a text file used to store tabular data
- Each line is a row
- Each value is a field (column)

Example:

Company,Price,Change

Alpha,120,5

Beta,98,-2

Gamma,143,1

Note:

CSV files can be created and edited using any text editor such as Notepad or Excel



Why Work With CSV Files?

CSV files are commonly used to:

- Export spreadsheet data
- Exchange data between programs
- Store simple tables
- Analyze records in Python

Examples: sales data, inventory reports, survey results, stock watchlists

Python Libraries: Module, Package, Library

A **module** is a Python file with code

A **package** is a collection of modules

A **library** is a larger collection of tools for a purpose

Example: **pandas** is a library for working with tabular data

Suppose you have a file called `math_tools.py`, which defines:

```
def add(a, b):  
    return a + b
```

In another file:

```
import math_tools  
print(math_tools.add(2, 3)) # Output: 5
```

Why Pandas?

Pandas helps us:

- Read CSV and Excel files
- Organize data into tables
- Filter and sort records
- Convert data types
- Compute statistics
- Save results back to files
- Visualize datasets

Installing pandas on PyCharm (mac):

PyCharm -> Settings

Python -> Interpreter

+

Pandas

install

Reading CSV Files into DataFrames

Importing Pandas

```
import pandas as pd    # pd is just a short alias for pandas
```

use read_csv()

```
catalog = pd.read_csv("MarketTrend.csv")
```

```
print(catalog)
```

What Is a DataFrame?

A dataframe is:

- A table of rows and columns in Python.
- row = one record
- column = one field

Inspecting a DataFrame

```
print(catalog.head())    # head() shows first 5 rows by default, you can specify head(3), etc.
```

```
print(catalog.columns)  # columns shows column names
```

```
print(catalog.dtypes)   # dtypes shows data types
```

```
print(catalog["Price"]) # single brackets → one column
```

```
print(catalog[["Company", "Price"]]) # double brackets → dataframe with multiple columns
```

Most Common Data Frames Attributes

Attribute	Description	Example
<code>df.columns</code>	Returns the column names of the DataFrame	<code>df.columns</code>
<code>df.index</code>	Returns the row labels (index)	<code>df.index</code>
<code>df.shape</code>	Returns number of rows and columns (rows, columns)	<code>df.shape</code>
<code>df.dtypes</code>	Shows the data type of each column	<code>df.dtypes</code>
<code>df.size</code>	Returns total number of elements in the DataFrame	<code>df.size</code>
<code>df.ndim</code>	Returns number of dimensions (always 2 for DataFrames)	<code>df.ndim</code>
<code>df.values</code>	Returns the raw data as a NumPy array	<code>df.values</code>

Most Common Data Frames Methods

Method	Description	Example
<code>df.head()</code>	Displays the first 5 rows of the DataFrame	<code>df.head()</code>
<code>df.tail()</code>	Displays the last 5 rows of the DataFrame	<code>df.tail()</code>
<code>df.info()</code>	Shows summary information about the DataFrame (columns, data types, non-null values)	<code>df.info()</code>
<code>df.describe()</code>	Generates statistical summary of numeric columns	<code>df.describe()</code>
<code>df.sort_values()</code>	Sorts the DataFrame by a column	<code>df.sort_values(by="Company")</code>
<code>df.groupby()</code>	Groups rows by column values for aggregation	<code>df.groupby("Change").min()</code>
<code>df.mean()</code>	Calculates the average of numeric columns	<code>df["Price"].mean()</code>
<code>df.sum()</code>	Calculates the total of numeric columns	<code>df["Price"].sum()</code>
<code>df.to_csv()</code>	Writes the DataFrame to a CSV file	<code>df.to_csv("output.csv")</code>
<code>df.to_excel()</code>	Writes the DataFrame to an Excel file	<code>df.to_excel("output.xlsx")</code>

DataFrame Index

Setting an Index

Every DataFrame has an **index**.

- identifies each row
- acts like a row label
- is used when combining or aligning data

If you try `print(catalog)`, the numbers on the left are the **index**

Sometimes a column uniquely identifies each record.

```
catalog = catalog.set_index("Company") # make Company the index
```

The groupby() Method

groupby() allows us to:

Split data into groups based on a column and perform calculations on each group.

Example question:

What is the **average price by company type**?

```
# create groups based on Company Type
```

```
catalog.groupby("CompanyType")
```

```
# groupby() must be followed by an operation like: mean(), sum(), count()
```

```
catalog.groupby("CompanyType")["Price"].mean() # select the Price column and compute average
```

```
catalog.groupby("CompanyType")["Company"].count() # how many companies belong to each type
```



Selecting Rows and Columns

Select rows using position

- Pandas allows selecting parts of a dataset.

`catalog.iloc[0:3]` # select first three rows

- Select columns by name

`catalog[["Company", "Price"]]`

- Filtering data

Filtering Data

We can filter rows using conditions.

Example: show only **Technology** companies

```
catalog[catalog["CompanyType"] == "Technology"]
```

Example: companies with **Price > 100**

```
catalog[catalog["Price"] > 100]
```

Combining both: **Technology** companies with **Price > 100**

```
catalog[(catalog["CompanyType"] == "Technology") & (catalog["Price"] > 100)]
```

Sorting Data

We can sort rows by a column.

Sort by price

```
sorted_c = catalog.sort_values(by="Price")
```

Sort from highest to lowest

```
sorted_c = catalog.sort_values(by="Price", ascending=False)
```

Combining DataFrames

- Sometimes we have **multiple datasets**.
- We may want to: combine datasets, update values, or add new rows.

Method	Purpose
<code>concat()</code>	stack datasets together
<code>merge()</code>	combine datasets using matching columns
<code>update()</code>	update existing values

Data Set A

Company	Price
Alpha	120
Beta	98

Data Set B

Company	Price
Beta	101
Gamma	143

Common DataFrame Combination Methods

concat

concat() does not check for duplicates

`pd.concat([df1, df2])`

Company	Price
Alpha	120
Beta	98
Beta	101
Gamma	143

Data Set A

Company	Price
Alpha	120
Beta	98

Data Set B

Company	Price
Beta	101
Gamma	143

Common DataFrame Combination Methods

update

First set the **Company** column as the index so pandas knows how to match rows

```
df1 = df1.set_index("Company")
```

```
df2 = df2.set_index("Company")
```

```
df1.update(df2) # df1 will be updated  
print(df1)
```

Company	Price
Alpha	120
Beta	101

The value for **Beta** is updated from **98** → **101**

Gamma is not added because `update()` only modifies **existing** rows

Common DataFrame Combination Methods

merge

merge() combines rows based on a **matching column**

```
Merged = pd.merge(df1, df2, on="Company", how="outer")
```

- Rows are matched using the **Company** column
- Price_x comes from the first dataset
- Price_y comes from the second dataset
- Gamma appears even though it only exists in Dataset B because we used **how="outer"**

Company	Price_x	Price_y
Alpha	120	NaN
Beta	98	101
Gamma	NaN	143

Option

"inner"

"left"

"right"

"outer"

Meaning

Keep only rows that exist in **both datasets**

Keep **all rows from the first dataset**

Keep **all rows from the second dataset**

Keep **all rows from both datasets**

Problem After merge()

After merging two datasets we may get duplicate columns.

`combine_first()`

Keep the **new value** if it **exists**, otherwise keep the old value.

```
df3["Price"] = df3["Price_y"].combine_first(df3["Price_x"])
```

Company	Price_x	Price_y
Alpha	120	NaN
Beta	98	101
Gamma	NaN	143

Final Step

Keep only the desired columns

```
df3 = df3[["Company", "Price"]]
```

What Are Missing Values?

Sometimes datasets contain **missing or incomplete data**

In pandas, missing values are usually represented as: NaN (Not a Number)

We can check for missing values using

`catalog.isnull()`

To see how many missing values exist in each column:

`catalog.isnull().sum()`

Removing Missing Values

We can remove rows that contain missing values.

```
catalog.dropna()
```

Remove Columns with Missing Values

```
catalog.dropna(axis=1) # This removes entire columns that contain missing data.
```

Remove Rows Only If ALL Values Are Missing

```
catalog.dropna(how="all") # This removes rows only if every column is NaN.
```

Instead of deleting rows, we can **fill missing values**.

Example: replace missing values with 0.

```
catalog.fillna(0)
```



How Pandas Handles Missing Values in Calculations

When using `sum()`:

Missing values (NaN) are treated as **zero**

```
print(catalog["Price"].sum())
```

When using `groupby()`:

Rows containing missing values in the grouped column are **excluded** from the groups

Rows where `CompanyType` is missing **will not be included in any group.**

```
df.groupby("CompanyType")["Price"].mean()
```

What Are Aggregation Functions?

Aggregation functions summarize data by computing a **single value** from a group of values.

Function	Description	Example
<code>mean()</code>	Average value	<code>df["Price"].mean()</code>
<code>sum()</code>	Total of values	<code>df["Price"].sum()</code>
<code>count()</code>	Number of non-missing values	<code>df["Price"].count()</code>
<code>min()</code>	Smallest value	<code>df["Price"].min()</code>
<code>max()</code>	Largest value	<code>df["Price"].max()</code>

Aggregation Example

average price by company type

```
catalog.groupby("CompanyType")["Price"].mean()
```

We can apply multiple aggregation functions at once.

```
catalog.groupby("CompanyType")["Price"].agg(["mean","min","max"])
```

Common Type Conversions

When data is read from CSV files, numbers may be stored as **strings (text)**

Company	Price
AlphaCorp	"120"
BetaTech	"98"

If the column is text, calculations may not work correctly.

We can convert a column to another type using `astype()`

Conversion	Example
String → Integer	<code>astype(int)</code>
String → Float	<code>astype(float)</code>
String → Numeric	<code>pd.to_numeric()</code>
Integer → String	<code>astype(str)</code>

```
catalog["Price"] = catalog["Price"].astype(float)
```



How to Write to an Excel File

Writing Excel files requires installing a helper library like **openpyxl** or **xlsxwriter**.

Install via PyCharm GUI (Mac):

- Open PyCharm
- Go to PyCharm → Settings
- Click **Python Interpreter**
- Click +
- Search for: **openpyxl** or **xlsxwriter** and install

index=False prevents pandas from writing row numbers

```
df.to_excel("output.xlsx", index=False)
```

Unit 6 Course Module



READING



DISCUSSION



ASSIGNMENT



SEMINAR



Any Questions?

<https://pandas.pydata.org/docs/>